

Financial Hub — Onboarding, Plan Assignment & Scoring Redesign

Status: **Draft for team review**. Written after a full code-level audit of the current app (see `BACKEND_FRONTEND_AUDIT.md` / `ROADMAP.md` Phase A for the earlier bug pass) plus external research into money psychology, income-persona segmentation, category-prioritization frameworks, and habit-loop gamification design. This doc is the concrete redesign plan requested for onboarding + plan assignment + the discipline/streak scoring system, built on the answers the team gave to the 8 scoping questions.

Part 1 — What the research says (summary, for context on the recommendations below)

1.1 Money personality types (behavioral finance / financial therapy literature)

The most consistently cited framework across financial-therapy sources (Olivia Mellan's *The Advisor's Guide to Money Psychology*, and multiple independent write-ups) groups people into a small set of recurring money personalities:

- **Spender** — spends as an emotional/identity act, struggles to delay gratification, avoids budgeting language.
- **Saver** — saves out of a fear-of-scarcity motivation; can under-spend even when it's not needed.
- **Avoider** — anxious about money, avoids checking balances or making financial decisions at all; can be a spender or saver underneath the avoidance.
- **Amasser / Hoarder** — motivated by the *feeling* of having money available, not by what it can buy or protect.
- **Money Monk** — uncomfortable with having money at all; may sabotage their own financial security out of guilt.

This is a useful **lens for tone and friction design**, not a replacement for the income-based personas below. A "Spender" and a "Saver" can both be salaried employees — the personality type should shape *how* the app talks to them (e.g. an Avoider needs low-friction, non-judgmental nudges; a Spender needs stronger default friction on discretionary reallocations), while the income persona shapes *what pockets and caps* they get. We recommend treating this as a **secondary modifier layer**, not the primary onboarding axis — see §2.3.

1.2 Income/earner personas relevant to your market

Kenya-specific research changes the picture from the current binary (salaried vs. freelancer):

- Kenya's gig/platform economy is large and growing fast — roughly **1.5M+ gig workers** and **~\$1B+ in annual market value**, with ride-hailing/delivery workers alone showing a wide spread: about a third earn under 25% of income from the platform, ~44% earn a quarter-to-three-quarters from it, and ~20% depend on it for over 75% — i.e. "gig

worker" is not one income pattern, it's a *spectrum of income concentration*, which matters for how confidently the app can predict their next payday.

- Kenya's financial life runs overwhelmingly on **mobile money (M-Pesa)**, not bank cards/statements — any future "smart" fixed-expense detection should be designed around SMS/mobile-money-shaped data, not bank statement PDFs.
- Nearly all working Kenyans are effectively informal-sector-adjacent even when they have "formal" jobs — side income, family obligations (school fees, upkeep for relatives), and irregular top-ups are common even for salaried earners, not just freelancers.

This supports going beyond the current 2-pattern model. See §2.1 for the proposed persona set.

1.3 Category prioritization research (needs vs. wants vs. savings)

The dominant framework here is the **50/30/20 rule** (Elizabeth Warren): 50% needs, 30% wants, 20% savings/debt, of after-tax income. Every source agrees on two things that matter for your redesign:

- The percentages are explicitly **a starting point, not a law** — real households, especially lower-income ones, routinely find needs alone eat 70-80%+ of income, which breaks the 50% cap. Multiple sources (Ramsey, Forbes) explicitly criticize rigid fixed percentages for this reason.
- **Housing/rent is treated as its own headline "need"** in every framework we found, distinct from general utilities — none of the standard frameworks fold rent into a generic "utilities" bucket the way your current fixed-expense suggestions do.
- The savings category is consistently framed as **goal-inclusive** (emergency fund, a named goal, debt payoff), not a single undifferentiated "savings" line.

Recommendation: use 50/30/20 as a *reference shape*, not a hardcoded formula — your onboarding should derive the real needs/wants/savings split from the person's actual fixed-cost ratio (as your rules-engine partially already does), while flagging to the user when their needs ratio is unusually high (a real behavioral signal worth surfacing, not hiding).

1.4 Mental accounting (Thaler) — why "pockets" work at all

Richard Thaler's mental accounting research (Nobel-cited, 1999 and 1985 papers) is the actual theoretical basis for pocket-based budgeting: people don't treat money as fungible even though economically it is — they mentally file it into non-fungible categories ("rent money," "fun money") and this **self-imposed non-fungibility is what makes budgets work at all**, going back to the literal cash-envelope system. The implication for your product: the *friction* between pockets (reallocation cooling-off, locked fixed pockets) isn't a UX inconvenience layered on top of the "real" product — it **is** the product. This validates the direction the team is already taking with locked, single-purpose fixed pockets (§3).

1.5 Streak/habit-loop gamification (Duolingo and similar)

Key, consistently-replicated findings across multiple design write-ups on Duolingo's streak system:

- Streaks work primarily through **loss aversion** — once a streak exists, the fear of losing it is a stronger motivator than the reward of extending it. Reported internal figures credit streaks with large jumps in daily engagement (commonly cited: streaks increasing daily commitment substantially, on the order of 50%+ in some breakdowns — treat exact percentages as directional, not verified).
- Effective streak systems have **forgiveness built in** (a "freeze"/grace mechanic) — a single missed day should not zero out weeks of progress, or the system creates anxiety and compulsive-use risk instead of healthy habit formation. This is explicitly flagged across sources as an ethical requirement, not just a retention tactic.
- **Milestones should be layered at increasing intervals** — an early win (day 3), a first real achievement (day 7), habit formed (day 30), identity-level commitment (100+).
- The most durable retention state isn't loss-aversion at all — it's **identity formation** ("I am someone who does this"), which the more mature systems build toward once the loss-aversion phase has hooked the user.
- Reminder notifications work best when **triggered by the user's own behavior pattern** (e.g. "you usually log spend around 6pm and haven't today"), not a fixed broadcast time.

This directly informs the two-layer scoring architecture in Part 4.

Part 2 — Redesigned onboarding

2.1 Persona set (replaces the current 2-pattern income model)

Current code only knows `(salaried | mix | freelancer)`. Proposed expansion, grounded in the research above:

Persona	Signal	Why it needs distinct handling
Student	No/minimal income, or a stipend/allowance pattern; often flagged directly	Very low absolute daily amounts, few/no fixed bills, spending habits are the dominant signal — not income stability
Salaried — single employer	Regular, predictable, one source	Current model's strength; keep close to what exists today
Salaried — with side income	Regular base + irregular top-ups	Very common in Kenya per research above; needs a "stable core, flexible edge" split rather than being forced into pure-salaried or pure-freelancer
Gig/platform worker (high income concentration)	Ride-hailing/delivery-style, >50-75% of income from one or few platform sources	Volatile day-to-day, but has <i>some</i> predictability (weekly patterns, platform payout cadence) — different from a freelancer with genuinely lumpy client-based income
Freelancer / project-based	Irregular multi-client income, current model's "freelancer"	Keep, but split cleanly from gig-platform workers above — the runway/cadence math is genuinely different (client invoice cycles vs. daily/weekly platform payouts)
Informal / self-employed trader	Business-linked personal income (small shop, boda boda owner-operator, services)	Needs categories your current model doesn't have (stock/restock cost is a "fixed-ish" recurring cost, not a discretionary spend) — flagged here for awareness even though full MSME support stays out of scope per the roadmap; the <i>individual</i> version of this persona (a solo trader managing personal money) is in scope now

Recommendation: don't try to build 6 fully bespoke plan-generation paths at once. Structure the persona as **two independent dimensions** instead of one big enum:

1. **Income stability** (predictable / semi-predictable / volatile) — drives runway math and daily-cap adaptiveness (this is what your current salaried/mix/freelancer really encodes).
2. **Life stage / structure** (student / working adult / self-employed) — drives which fixed-expense suggestions, pocket names, and default categories get offered.

This keeps the rules-engine combinatorics sane (3×3 , not a hand-built tree per named persona) while still letting onboarding *feel* like it recognizes "you're a student" or "you're a Bolt driver" specifically in its copy and suggestions.

2.2 Deeper onboarding question set

Current: 4 data points (income pattern, spending habit, income amount, fixed total).

Proposed additions, each justified by what it unlocks downstream:

1. **Household/dependents** — "Do you regularly send or spend money for others (school fees, family upkeep, dependents)?" → feeds a non-judgmental default pocket ("Family & obligations") instead of forcing this spend into "Other," and feeds the needs-ratio calculation (§2.4).
2. **Existing savings / emergency buffer** — "Do you already have money set aside you could live on for a while if income stopped?" (banded: none / less than a month / 1-3 months / 3+ months) → makes the savings target and lock length actually personalized (§4) instead of a flat 10%.
3. **Savings goal** — what for, roughly how much, roughly by when (§4 — this is the concrete answer to "goal-driven savings, minimum 5%").
4. **Money personality self-check** (2-3 short, non-clinical questions drawn from the Spender/Saver/Avoider framing in §1.1, phrased behaviorally not diagnostically — e.g. "When you get unexpected money, what do you usually do?") → sets the *tone/friction modifier* described in §2.3, not the plan type itself.
5. **Income concentration** (for anyone not clearly salaried) — "Does most of your money come from one place, or a few different places?" → distinguishes gig-platform-style volatility from true multi-client freelance lumpiness, both currently collapsed into one freelancer bucket.

This is a longer onboarding than today's 3-screen flow — acceptable per your answer to go deep, but it should still be paced (multi-step, save-as-you-go, skippable-with-defaults for anything not load-bearing) so completion rate doesn't collapse. We'd recommend usability-testing the length before committing to a final screen count.

2.3 Money-personality as a modifier layer, not a plan-type driver

Concretely: keep plan assignment (§2.5) driven by income stability + fixed-cost ratio + persona, as today — but let the money-personality answers set:

- **Cooling-off duration and framing** — an Avoider gets gentler, reassuring copy and possibly a longer default cooling-off; a Spender gets a firmer default (still never punitive, per your existing "supportive framing" rule).
- **Notification tone and cadence** — once notifications actually exist (see bug list, Part 6), a Saver doesn't need much nudging; an Avoider benefits from very low-friction, frequent, small check-ins rather than infrequent heavy ones.
- **Which insights get surfaced first** — a Spender benefits from seeing "days since last reallocation out of savings" prominently; a Saver might benefit more from seeing goal progress.

This avoids the trap of building a 5-6-way plan-type matrix that's hard to test and explain, while still making the app feel personality-aware.

2.4 Real, revised needs/wants/savings math (replaces the current flat 20%-of-income structured/daily cutoff)

This directly answers scoping question 4. Recommendation: replace the single hardcoded 20% threshold with a **derived needs ratio** using the same underlying idea as 50/30/20, adapted per §1.3's finding that fixed percentages don't hold at lower incomes:

- Compute $\text{needsRatio} = \text{fixedTotal} / \text{incomeAmount}$ (this already exists as $\text{remainingAfterFixed}$), just reframed).
- **Needs ratio $\geq \sim 70\%$** → Daily Budget, regardless of stated spending habit — at this ratio there usually isn't enough discretionary room left for meaningful category pockets, and tight daily caps are the more honest tool. (Today's code only checks the *habit* answer plus a flat 20% remainder cutoff — it doesn't weight how tight things structurally are.)
- **Needs ratio roughly 40-70%** → let the *behavioral* signal (spending habit answer + money-personality modifier) decide between Structured and Daily — this is the genuinely ambiguous middle band where personality matters most.
- **Needs ratio below $\sim 40\%$** → Structured is viable by default, but still check the spending-habit answer — a low needs ratio with a "caught off guard" habit answer should still lean Daily, since low fixed costs don't guarantee spending discipline.

This turns a single hardcoded cutoff into a small, inspectable decision table instead of a black box, keeps the "inferred, not chosen" principle from the PRD, and gives you a real "why this plan" explanation to work with (§2.6) — because now the reasoning is genuinely derived from *their* numbers, not picked from a sentence pool.

Open modeling question for the team: where exactly should the 70% / 40% bands sit for your actual user base? These are directional starting points from the research, not backtested against real user data yet — worth revisiting once you have even a small pilot cohort's real fixed-cost ratios.

2.5 Persona-shaped pocket generation (replaces the current always-3-even-split logic)

Today: every Daily/Structured plan gets exactly Food/Transport/Leisure, split evenly, no matter who you are — the PRD's promised Student/Salaried/Freelancer daily-pocket variety was never built (confirmed in the code audit). Proposed:

- **Student persona** → a single flat daily spendable cap (matches the PRD's original KES 300/day concept, but the *amount* should be derived from their actual stated income/allowance, not a hardcoded constant) rather than 3 sub-categories — students' spending is usually too undifferentiated for Food/Transport/Leisure splits to mean anything.
- **Working-adult personas (salaried, salaried+side income, gig)** → keep category pockets, but let category *names and counts* come from what the person actually told you about their life (dependents → "Family & obligations" pocket; no stated transport need e.g. remote worker → fold transport into a smaller share rather than an even third).

- **Freelancer/gig with volatile income** → keep the existing adaptive-runway daily cap logic (this part of the code is genuinely good), but apply it *within* whatever category shape the persona above produced, instead of always defaulting to Food/Transport/Leisure.

2.6 Real "why this plan" explanations (replaces the ~6-sentence canned pool)

Once §2.4's decision is numeric and derived (needs ratio, spending-habit weight, persona), the explanation can be **template-generated from the actual numbers** instead of picked from a fixed set of ~6 strings:

"Rent, bills and other fixed costs take up **68%** of what you told us you earn — that doesn't leave much room for category-based pockets to mean anything, so we're giving you **daily caps** instead. Because you said [X], we're keeping [category] tighter than the others."

This is a string-interpolation change plus a slightly richer reasons object from the rules engine (percentages, not just rule-IDs) — not a rebuild, but should be scoped as its own ticket since the current `(PlanAssignReason)` shape (`(rule)` + static `(reason)` string) needs to carry numeric context through to the client.

Part 3 — User-defined fixed pockets (replaces the current single lump "Fixed Expenses" pocket)

This directly implements scoping answer 3. Current state: the DB already stores itemized `(fixed_expenses)` (name, amount, due day, category) — but the onboarding pocket-creation logic (`(createPocketInputs)` in `(onboarding.service.ts)`) still collapses everything into **one** lump "Fixed Expenses" pocket with a single `(monthly_allocation)`. The itemized data exists and isn't used for pocket structure — this is the concrete gap to close.

3.1 Proposed model

- **Each fixed expense the user defines becomes its own pocket**, named and categorized by the user (e.g. "Rent," "Martin's school fees," "Netflix") — not forced into the current fixed 8-category enum, which (per the earlier bug audit) doesn't even have a proper housing/rent category today. This is also the direct answer to scoping question 8 ("we can't have the rent pocket and one does not pay rent") — because the pocket is whatever the user names it, there's no default "Rent" pocket forced on someone who doesn't pay rent.
- **Each fixed pocket carries its own due day** (already captured in `(fixed_expenses.due_day)`) and is **locked until that day**, not just generically "fixed and settled."
- **Single-purpose enforcement**: money in a "Rent" pocket can only be spent as rent — this reuses the existing merchant-category soft-block machinery (§3.2 below), just pointed at a user-defined pocket instead of a fixed enum category.

3.2 Early-access behavior (your exact spec, mapped onto existing mechanics)

You described: if the user tries to use the money before the due day, and persists, a cooldown applies, then unlock, then a discipline-point deduction, and it's recorded. This maps almost exactly onto the **existing time-lock mechanism** (`is_time_locked`) / (`lock_until`) / (`unlockPocket`) / discipline cost per day-remaining) that today is only used for the Savings pocket — the redesign is to **generalize it to every user-defined fixed pocket**, locked until its due day each cycle, rather than building a new mechanism:

- Attempt to spend from a locked fixed pocket before its due day → currently (`SpendService.checkSpend`) already blocks this outright (`pocket_time_locked`). Under the new spec this becomes the **trigger for the cooldown flow** instead of a flat block — same pattern as the essential→leisure reallocation cooldown (§3.4 of the PRD), reused here.
- If the user pushes through past the cooldown → reuse (`unlockPocket`)'s existing discipline-cost-by-days-remaining math, and the existing behavior-event logging — this part of the code is already solid, it just needs to be reachable from a spend attempt on *any* fixed pocket, not only from the dedicated time-lock screen.
- **Category enforcement on unlock**: once unlocked, spend from that pocket should still be restricted to its own stated purpose (rent pocket → rent-category payees only) — this reuses the merchant soft-block/self-classify flow (§3.5 of the PRD) rather than needing new logic.

3.3 Dynamic, behavior-driven suggestions (not the current static/miscategorized list)

Today's (`getFixedExpenseSuggestions`) returns a hardcoded 8-item list with real miscategorization bugs (Internet/Mobile Data tagged "transport," School Fees tagged "healthcare" — see Part 6). Per your instruction, this needs to be **clean and dynamic based on behavior**, and per scoping answer 7, **no mock data** — this should be built for real, not simulated, ahead of the real bank/SACCO integration you mentioned is coming soon.

Proposed direction (to be scoped in detail once bank/SACCO partnership data access is defined):

- Short term (before partner data exists): suggestions are generated from **what the user has already told the app** — persona (a student won't be suggested "Rent" by default; someone who flagged dependents gets a "school fees / family" suggestion), and from their own edit history (if they've renamed or added a pocket type, similar users' onboarding defaults should reflect that pattern over time) — this is "dynamic" in the sense of being data-informed rather than a fixed list, without needing real bank data yet.
- Medium/long term (once partner bank/SACCO data access exists, per your note that real integration is coming): suggestions and even amounts can be inferred from real recurring-payment patterns in the linked account — this is the natural home for the "smart" detection the PRD always intended (§3.1 step 3, §7's "detection can be mocked/simulated" line — which we're explicitly *not* doing per your instruction, opting to wait for the real integration instead).

Open question for the team: what's the realistic timeline for the bank/SACCO partnership data access? That timeline determines whether "dynamic suggestions" ships as the behavior-informed version now, or waits for the real-data version — worth a short discovery/timeline conversation before this becomes a sprint commitment.

Part 4 — Goal-driven savings (replaces the flat 10%-of-remainder, 30-day-lock-for-everyone model)

Per scoping answer 6: what, when, for whom, with a derived rate, minimum 5%.

4.1 Proposed onboarding capture

- **What:** free-text or short-list goal name (emergency fund / a named purchase or event / a dependent's education / other) — matches PRD §5.2's "goal-based savings" promise, currently entirely uncaptured.
- **When:** target date or a rough timeframe band (similar pattern to the existing `incomeIntervalBand` banding for freelancers — exact dates are unreliable, bands are honest).
- **For whom:** self or named dependent (ties into the household/dependents question from §2.2) — mostly for framing/copy ("saving for Amara's school fees" reads very differently from a generic "Savings" pocket), not a technical access-control feature.

4.2 Derived rate, minimum 5%

Replace the current hardcoded `(MIN_SAVINGS_RATE = 0.10)` with a **floor, not a fixed rate**:

- **Absolute floor:** 5% of income after fixed costs (per your instruction) — never lower, regardless of goal size or needs ratio.
- **Derived target rate**, above the floor, computed from $\text{goal amount} \div \text{time-to-target} \div \text{available (income - fixed costs)}$ — i.e. work backward from what the user said they want and by when, the same way a savings calculator would, then compare that derived rate against their actual capacity:
 - If the derived rate is achievable within capacity → use it (this replaces the flat 10%).
 - If the derived rate would exceed a sane share of what's left after fixed costs → **don't silently force it** — surface it back to the user at the "preview the split" step (already an existing onboarding step per PRD §3.1.5) as "at your current income, this goal would take about **N months longer** than you hoped, or would need roughly **X%** of your spendable income — want to adjust the goal, the timeline, or the rate?" This keeps the promise honest instead of either failing silently or overcommitting the user.
 - If there's no stated goal (user skips it), fall back to the 5% floor plus whatever the existing needs-ratio-based calculation already produces as a sensible default — never leave savings at literally the floor with no reasoning shown.
- **Lock length becomes goal-derived too** — a 3-month emergency-buffer goal shouldn't default to the same 30-day lock as a 2-year goal. Suggested: lock length

scales with the stated timeframe (e.g. lock renews in step with the goal's own cadence), with the same early-unlock cooldown-then-cost mechanic as today, generalized per §3.2.

Part 5 — Category prioritization (answers scoping question 8)

You asked specifically how people prioritize categories, and noted the "can't force a Rent pocket on someone who doesn't pay rent" constraint — which the user-defined fixed pockets model in Part 3 already solves structurally. What's still open is **how to order/prioritize category suggestions and default spendable-pocket categories** for people who don't want to build everything from scratch.

Research-backed ordering (from §1.3, adapted, not copied verbatim from any one framework):

1. **Housing** (rent/mortgage) — near-universally the largest single fixed line item where it applies; needs to be a real, distinct pocket category (not folded into "utilities" as today), but only ever *suggested*, never forced, so it doesn't apply to someone who doesn't pay rent.
2. **Utilities** (power, water, basic connectivity) — genuinely fixed, small individually, but recurring and non-optional.
3. **Food/groceries** — universal need, but partially discretionary in *how much* is spent even if never zero — good fit for a spendable pocket with a cap, not a fixed pocket.
4. **Transport** — same shape as food: non-optional in total, discretionary in day-to-day amount.
5. **Family/dependents obligations** (school fees, upkeep sent to others) — should be elevated to a first-class, commonly-suggested category given how common this is in the Kenyan context per §1.2, rather than left to fall into "Other" as it likely does today.
6. **Health** — fixed where it's a known recurring cost (insurance/cover), spendable/unplanned otherwise.
7. **Personal/leisure & discretionary wants** — last in suggestion order, first to flex, matching every needs-vs-wants framework's ordering logic.
8. **Savings/goals** — not "last" in importance (the 50/30/20 literature is explicit that savings should be protected, not left as an afterthought), but last in this *category* list because it's handled by its own dedicated flow (Part 4), not the general suggestion list.

Personality-aware ordering: per your instruction ("according to the user personality"), the *order these are suggested in* during onboarding/pocket-setup should adapt to the money-personality modifier from §2.3 — e.g. a Saver might be shown savings/goal-setup earlier and more prominently, while a Spender or Avoider might benefit from seeing the concrete, tangible categories (rent, food) first, building up to savings once the basics feel handled rather than leading with it.

Part 6 — Bug fixes to land alongside this redesign (carried over from the full-app audit)

These aren't new work items introduced by this doc — they're confirmed, code-level bugs found while auditing the app for this plan, listed here so they get scoped into the same effort rather than lost:

1. **Merchant classification silently discards the chosen pocket.** `merchant_classifications` has no `pocket_id` column; the classify screen asks the user to pick a pocket and the answer is dropped server-side. Needs either a real schema column or the UI question removed until it's wired.
 2. **Reclassifying a transaction doesn't update it.** `MerchantService.classify`'s `transaction_updated` response is a fake echo — no DB write happens. The "Review and reclassify" flow doesn't actually fix the transaction category shown in the ledger.
 3. **Notifications are entirely inert.** Preferences (including "Savings milestones," directly relevant to the gamification work in Part 7) are stored but nothing anywhere sends a push — no `expo-notifications`, no scheduler. This blocks any streak-reminder or milestone-celebration mechanic from actually reaching the user outside the app.
 4. **Fixed-expense suggestion miscategorization.** Internet/Mobile Data tagged `transport` (should be `utilities`), School Fees tagged `healthcare` (should be `education`) — becomes moot once Part 3's user-defined model ships, but flag in case the old endpoint is still reachable during transition.
 5. **No `housing`/`rent` category exists in the schema at all** (`PocketCategorySchema` enum) — needs adding regardless of the fixed-pocket redesign, since it's referenced informally in suggestion copy today.
 6. **Daily rollover into Savings is a stub.** `calculateRollover()` in `home-store.ts` always returns 0; no backend job computes real rollover either. This is one of the most important reward events for the streak layer in Part 7 and should be prioritized alongside it, not treated as a separate unrelated bug.
 7. **`DEFAULT_SCORE` duplication risk** — currently safely re-exported from one file to another, but fragile; collapse to a single source before more code reads discipline-score defaults.
-

Part 7 — Scoring architecture (parking lot — for the follow-up conversation)

Not scoped in detail here since the team asked to map the flow and redesign onboarding first, but noting the connection points this redesign creates, so they're not lost:

- **Rollover-to-savings (bug #6 above)**, once fixed, becomes the natural trigger for the "streak" layer discussed earlier — a day where every spendable pocket stayed under cap is a real, honest daily win to celebrate, unlike anything currently wired.
- **Fixed-pocket due-day compliance** (Part 3) is a second, independent streak-worthy signal — "paid on time, N cycles running" — separate from daily spending discipline,

and arguably a stronger trust signal for the future lender-facing use case mentioned in the PRD's revenue-model notes.

- **Goal progress** (Part 4) gives a third, milestone-shaped signal (25%/50%/75%/100% of a named goal) that maps directly onto the "layered milestones at increasing intervals" pattern from the Duolingo research in §1.5.
- Per §1.5's forgiveness-mechanic finding, any streak system built on top of this should include a grace/freeze concept from day one, not bolt it on later — worth deciding this alongside the initial design rather than after user complaints.

Recommend a dedicated follow-up session to design this once Parts 2-6 above are agreed, since the scoring model should be built against the *real* event set this redesign produces (rollover, on-time fixed-pocket payments, goal milestones) rather than designed in the abstract first.

Open questions for the team (consolidated)

1. Final screen count/pacing for the deeper onboarding in §2.2 — needs a usability pass, not just a spec decision.
2. Where the needs-ratio bands (§2.4: currently proposed ~70% / ~40%) should actually sit — directional from research, not validated against your users yet.
3. Timeline for real bank/SACCO data access (Part 3.3) — determines whether dynamic fixed-expense suggestions ship in their behavior-informed form now or wait for the real-data form.
4. Exact set of money-personality questions and how literally to phrase them (§2.2 point 4) — needs to stay non-clinical and quick, worth a copywriting pass.
5. Whether "self or named dependent" savings-goal framing (§4.1) needs any privacy consideration before storing a dependent's name/relationship, given the app's existing PII-handling posture.